



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程就是运行中的程序。程序本身是没有生命周期的，只是存在磁盘上的一些指令 (或静态数据)
操作系统让这些字节运行起来，让程序发挥作用

```
0100010001010  
001001010101...
```

程序



进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

进程就是运行中的程序。程序本身是没有生命周期的，只是存在磁盘上的一些指令 (或静态数据)
操作系统让这些字节运行起来，让程序发挥作用





进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

进程就是运行中的程序。程序本身是没有生命周期的，只是存在磁盘上的一些指令 (或静态数据)
操作系统让这些字节运行起来，让程序发挥作用



程序本身作为一串二进制指令 / 数据无意义
操作系统让程序运行起来后，程序就可以发挥作用



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程的四个特征



1. 动态性：进程是程序的执行过程，动态性是进程最基本的特征
进程由创建而产生，由调度而执行，由撤销而消亡
所以进程有一定的生命期，程序只是静态的有序指令的集合

2. 并发性：多个进程共存于内存中，在一段时间内同时执行，所以并发也是进程的重要特征

3. 独立性：进程是一个能独立运行，独立获得资源，独立接受调度的基本单位

4. 异步性：进程都是按照异步方式运行的，即按各自独立、不可预知的速度向前推进



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

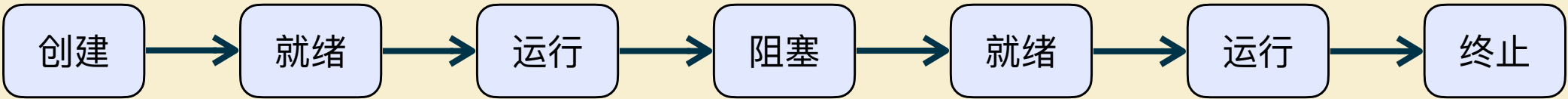
控制

通信

进程的四个特征

动态性

进程的生命周期



程序

```
#include <stdio.h>
int main() {
    char *str = "Beokayy_OS";
    printf("===== 开始打印 =====\n");
    for (int i = 1; i <= 5; i++) {
        printf("[%d] %s\n", i, str);
    }
    printf("===== 打印结束 =====\n");
    return 0;
}
```



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程的四个特征

并发性

CPU





进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程的四个特征

并行性

CPU





进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程的四个特征

并发性

CPU





进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程的四个特征

并发性

CPU





进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程的四个特征

并发性

CPU





进程

基本概念

内存映像

状态&转换

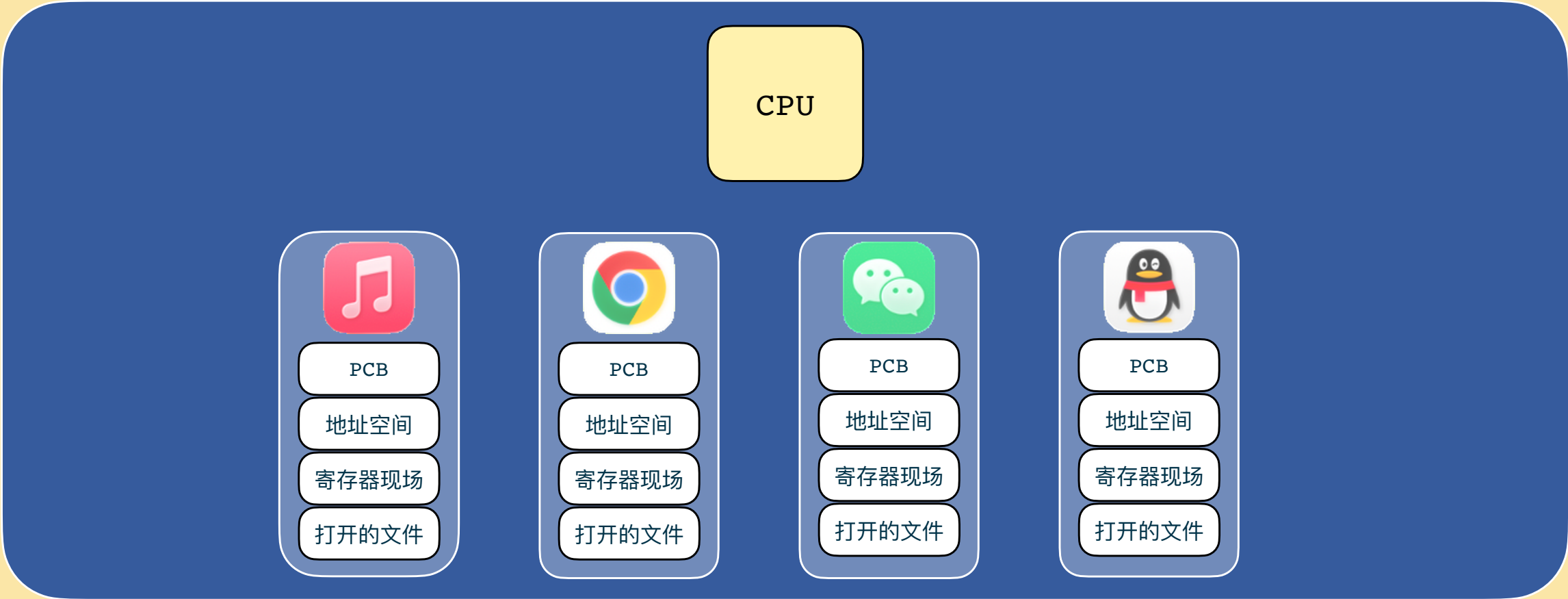
进程管理的数据结构

控制

通信

进程的四个特征

独立性



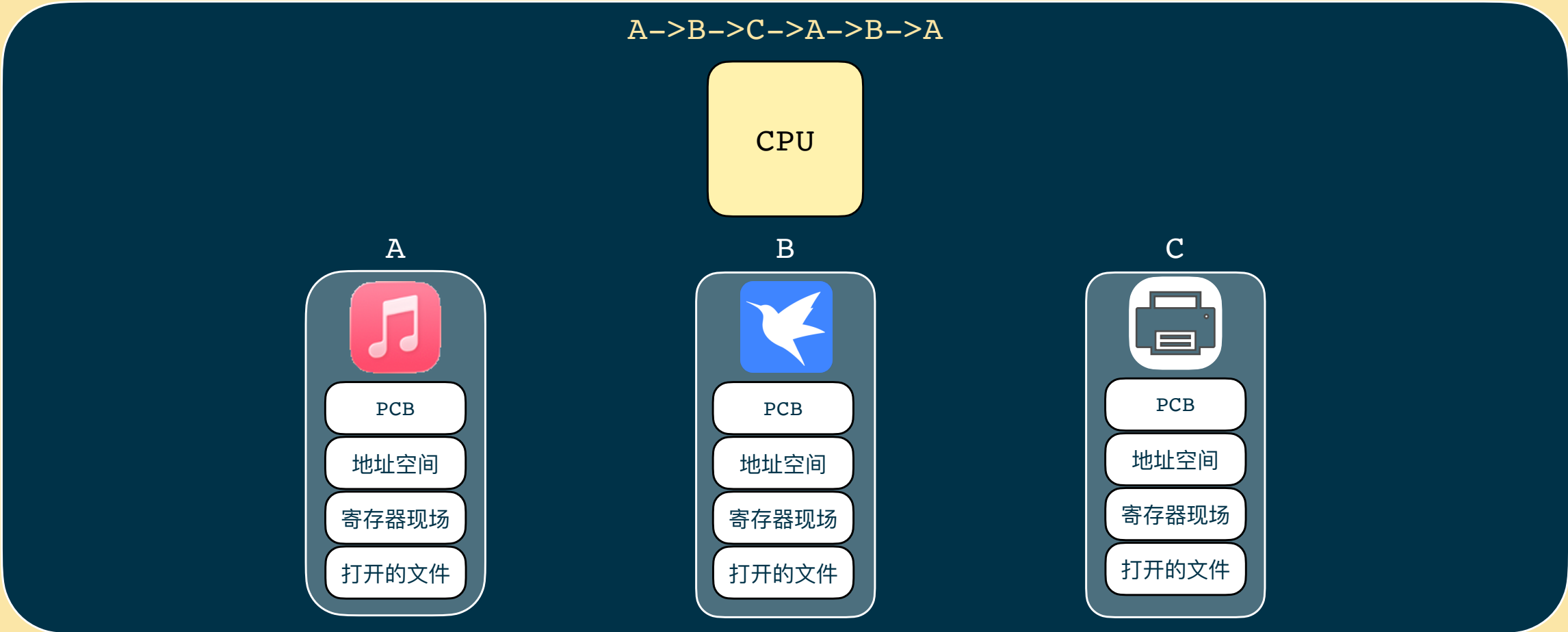


进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

进程的四个特征

异步性



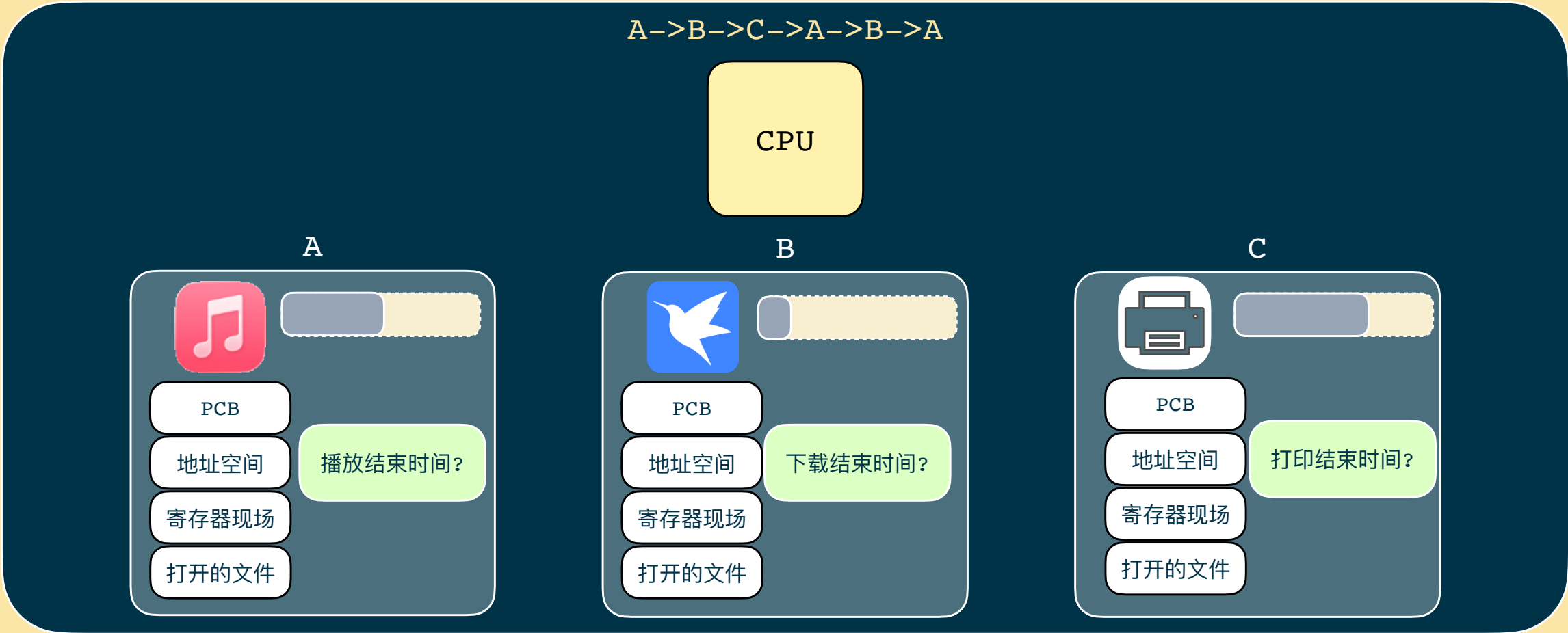


进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

进程的四个特征

异步性



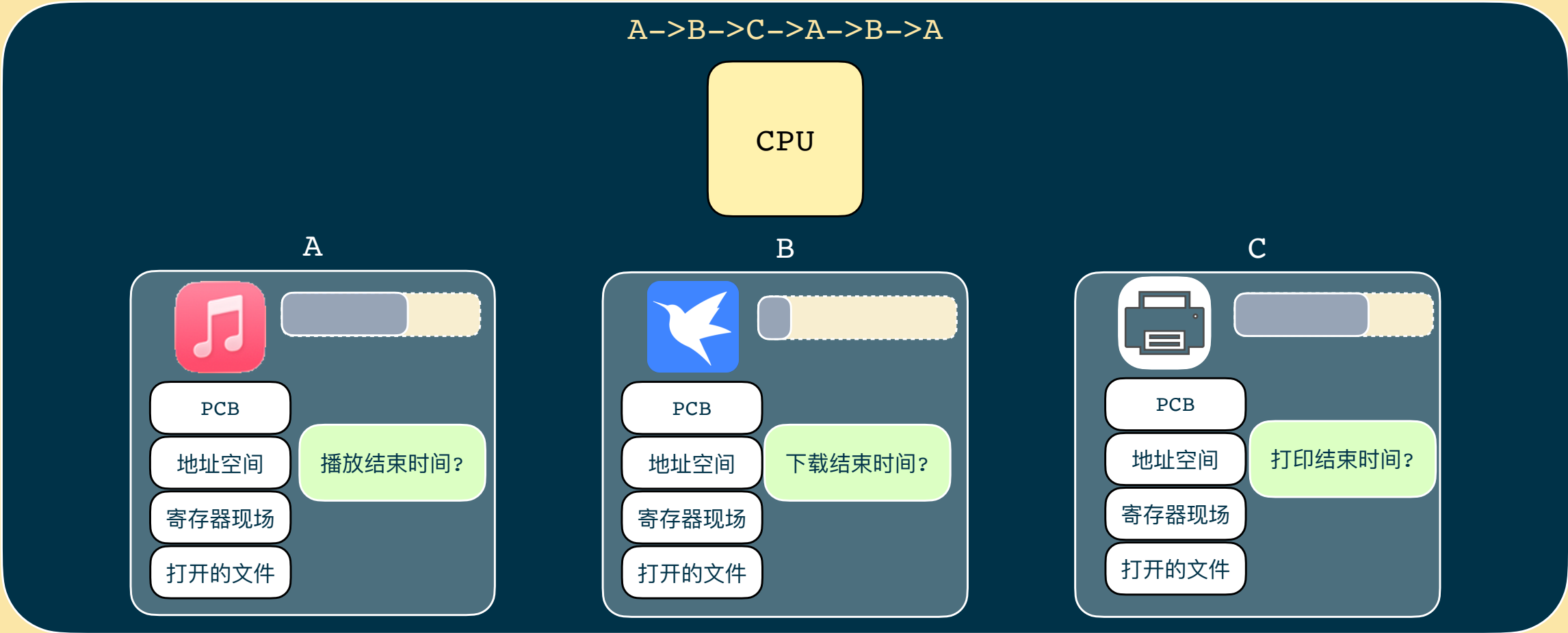


进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

进程的四个特征

异步性



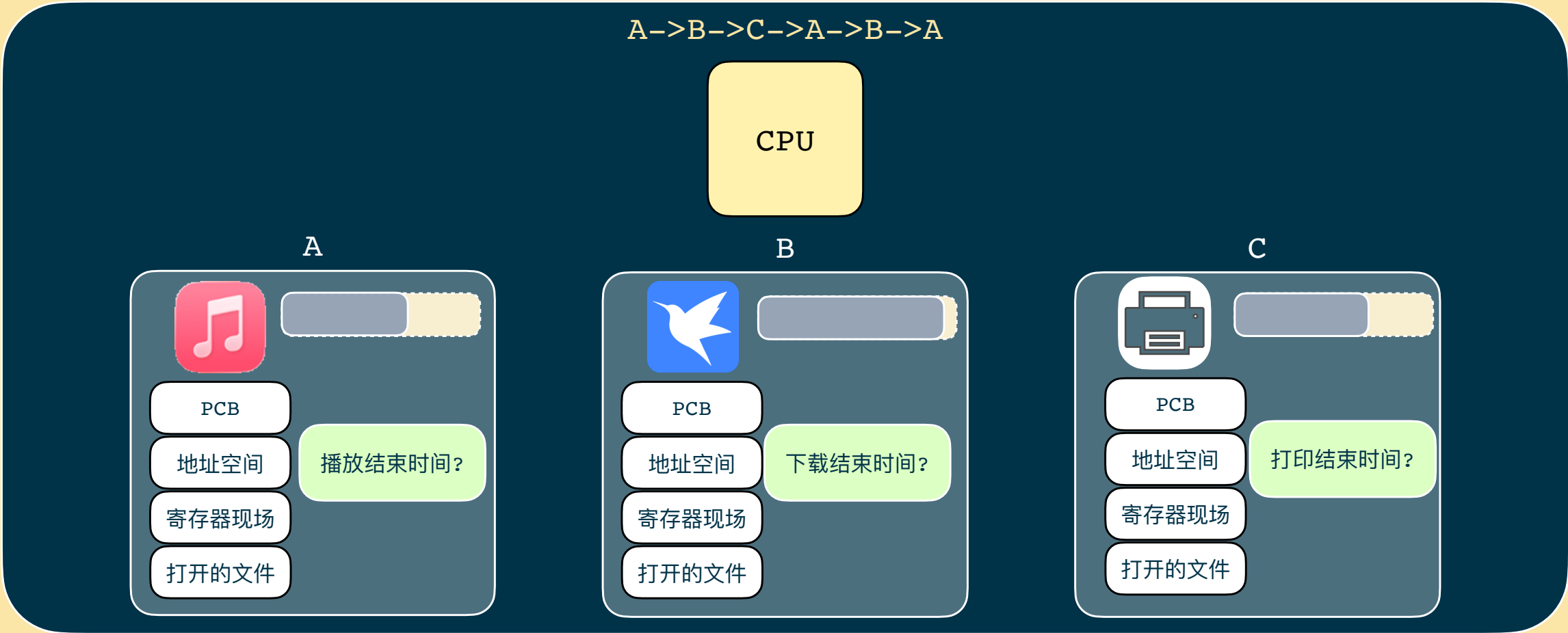


进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

进程的四个特征

异步性



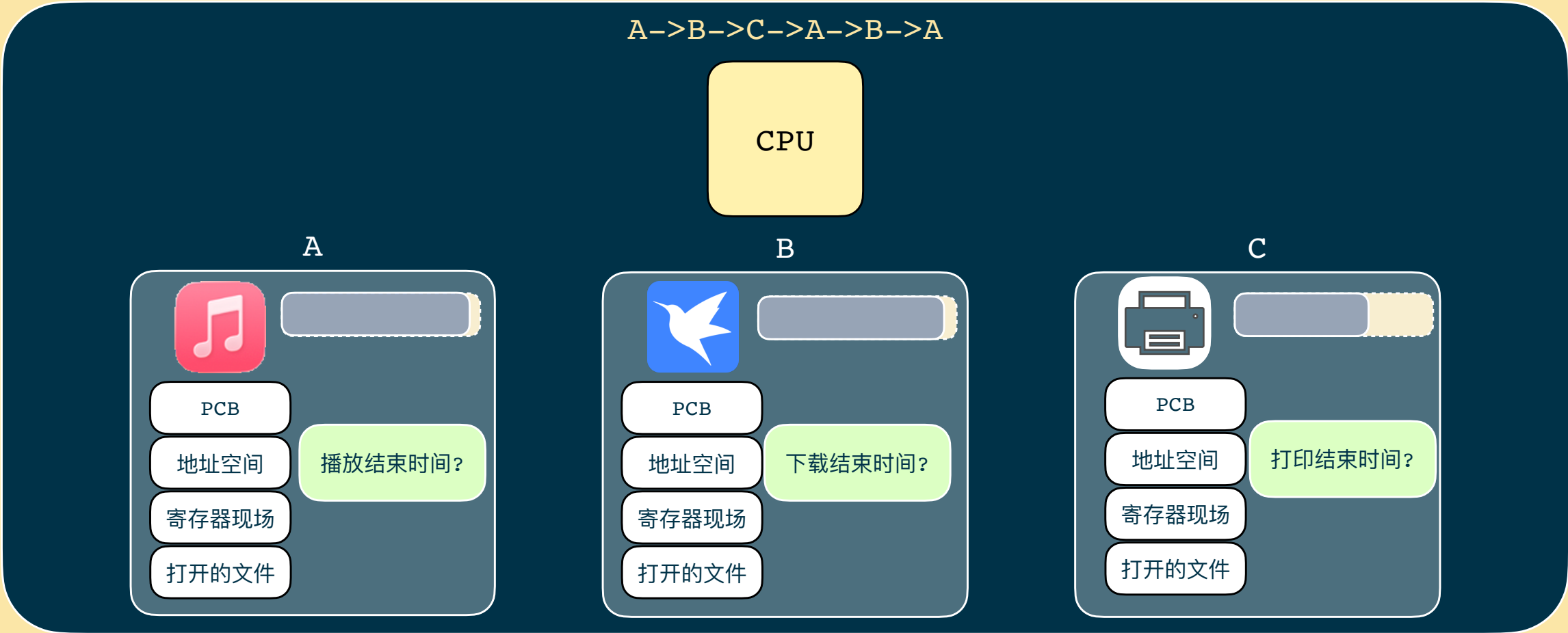


进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

进程的四个特征

异步性



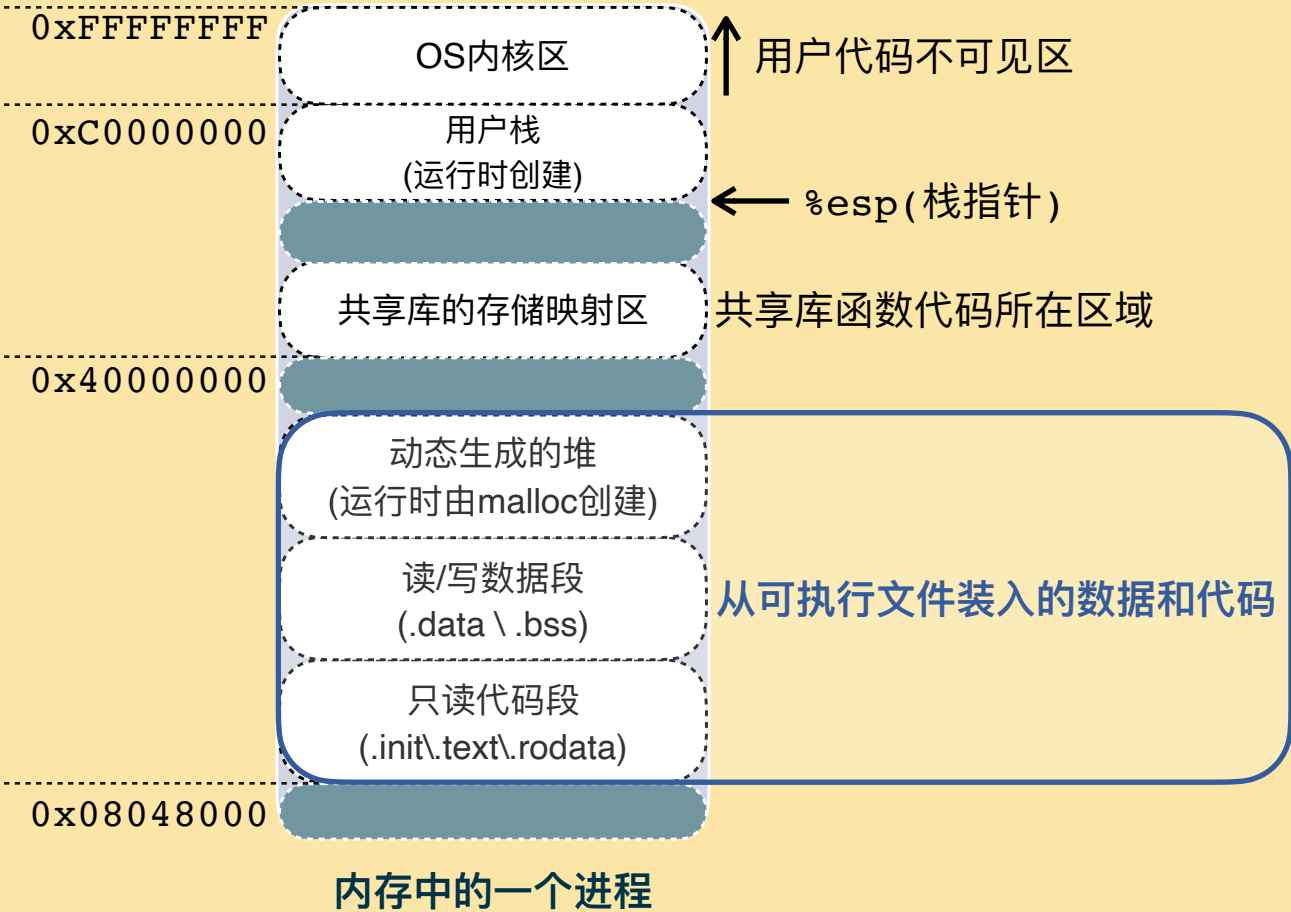


进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

请务必学习完虚拟存储器后再观看本部分

虚拟化技术：每个进程访问自己的地址空间，os会以某种方式将它映射到机器的物理内存中。
对于每个进程，进程都认为自己独占了整个CPU和整个虚拟内存空间



0xFFFFFFFF

OS内核区

↑ 用户代码不可见区

0xC0000000

用户栈
(运行时创建)

← %esp(栈指针)

共享库的存储映射区

共享库函数代码所在区域

0x40000000

动态生成的堆
(运行时由malloc创建)

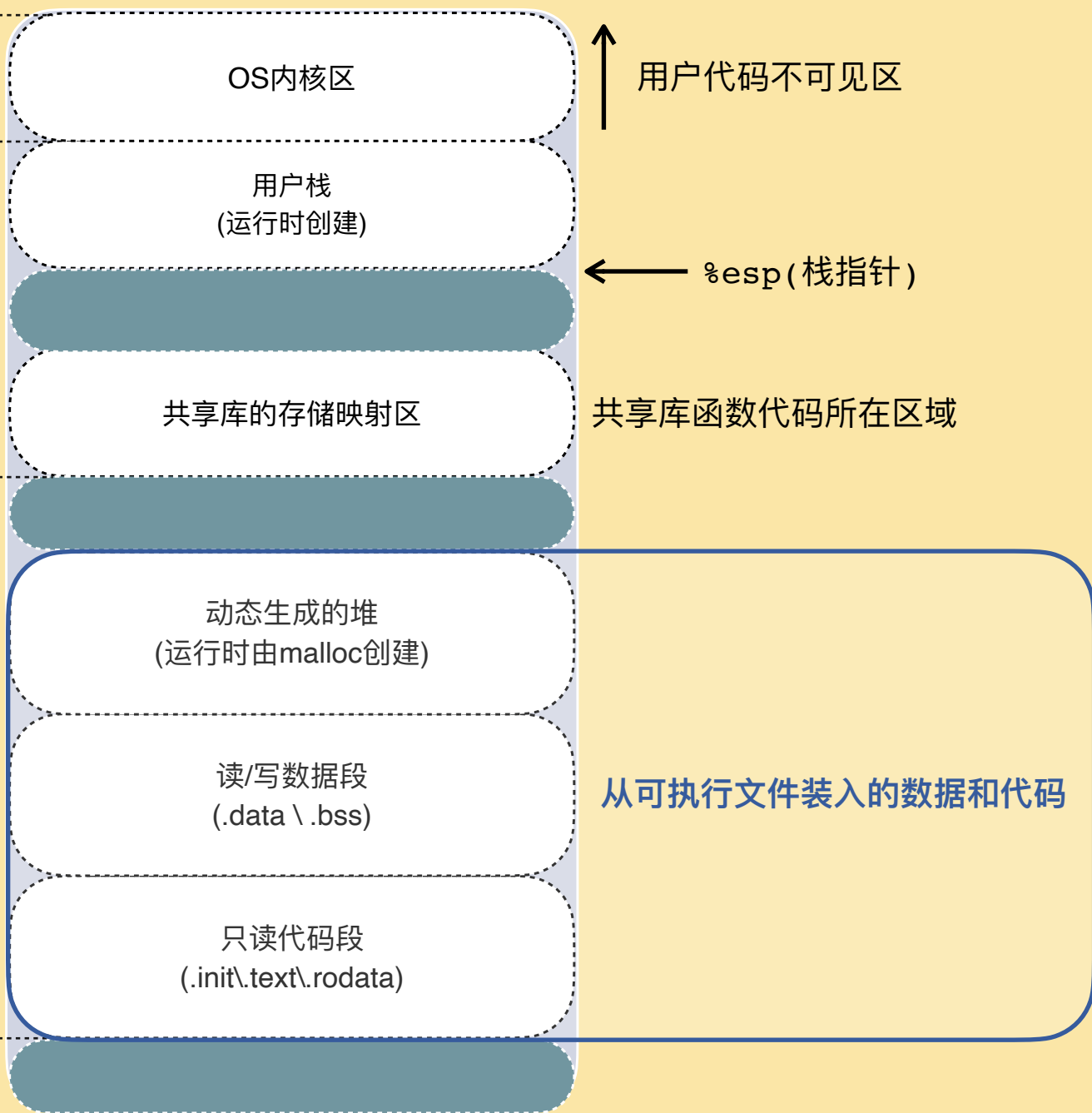
读/写数据段
(.data \ .bss)

从可执行文件装入的数据和代码

只读代码段
(.init\text\rodata)

0x08048000

内存中的一个进程

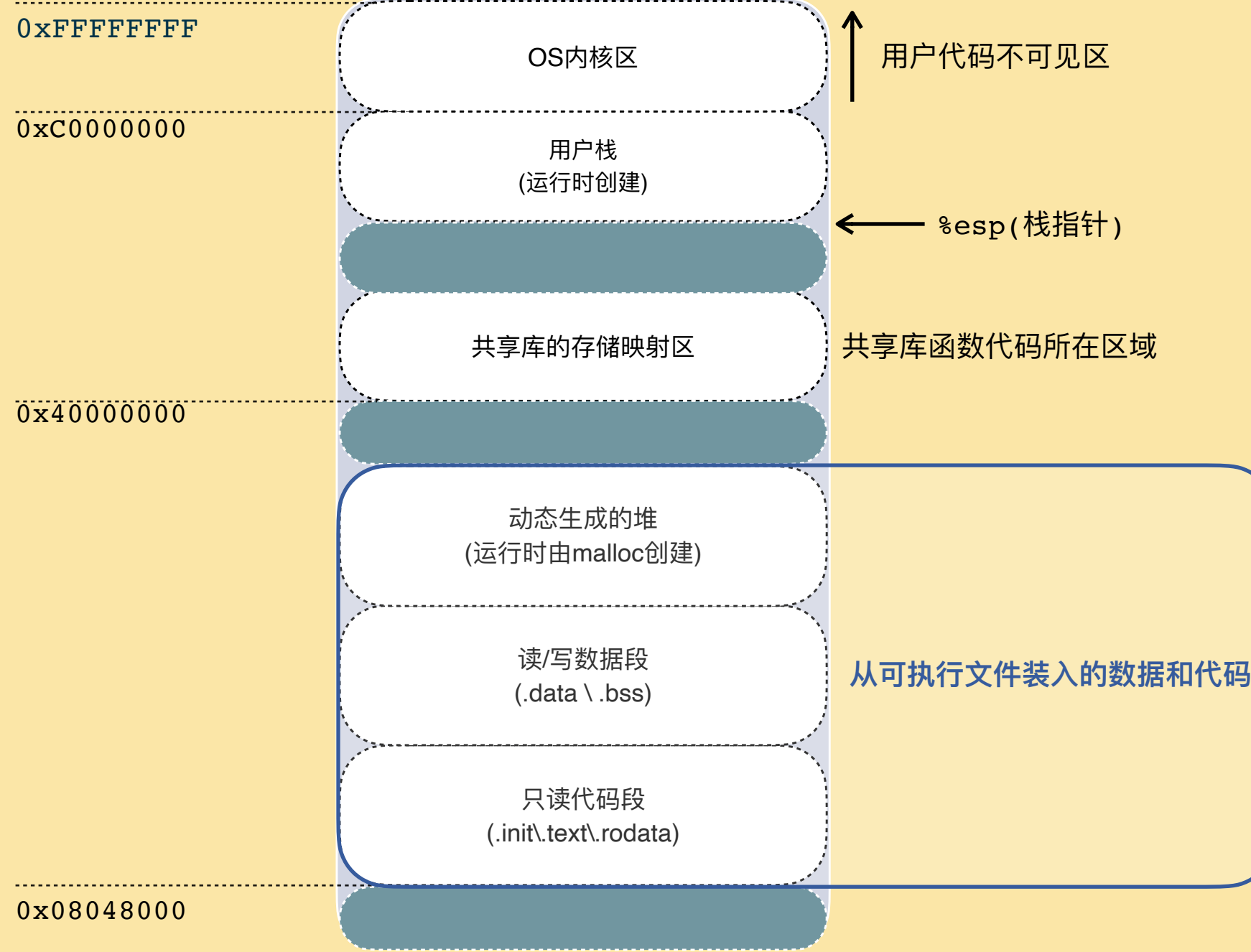


32位程序中虚拟地址为32个bit
即程序可以访问到的地址空间最大是
0xFFFF FFFF



32位程序中虚拟地址为32个bit
即程序可以访问到的地址空间最大是
0xFFFF FFFF

为每个进程创建巨大的、私有的虚拟
内存(virtualmemory)的假象
这种地址空间(addressspace)的抽
象让每个程序好像拥有自己的内存
而实际上操作系统秘密地让多个地址
空间复用物理内存(或者磁盘)



内存中的一个进程

程序调入内存运行时，就构成了进程的**内存映像**
一个进程的内存映像一般有几个要素：

代码段：即程序的二进制代码，代码段是只读的，可被多个进程共享

数据段：即程序运行时加工处理的对象，**包括全局变量和静态变量**

进程控制块PCB：放在系统区

堆：用来存放动态分配的变量，通过**调用malloc函数**动态的向**高地址**分配空间

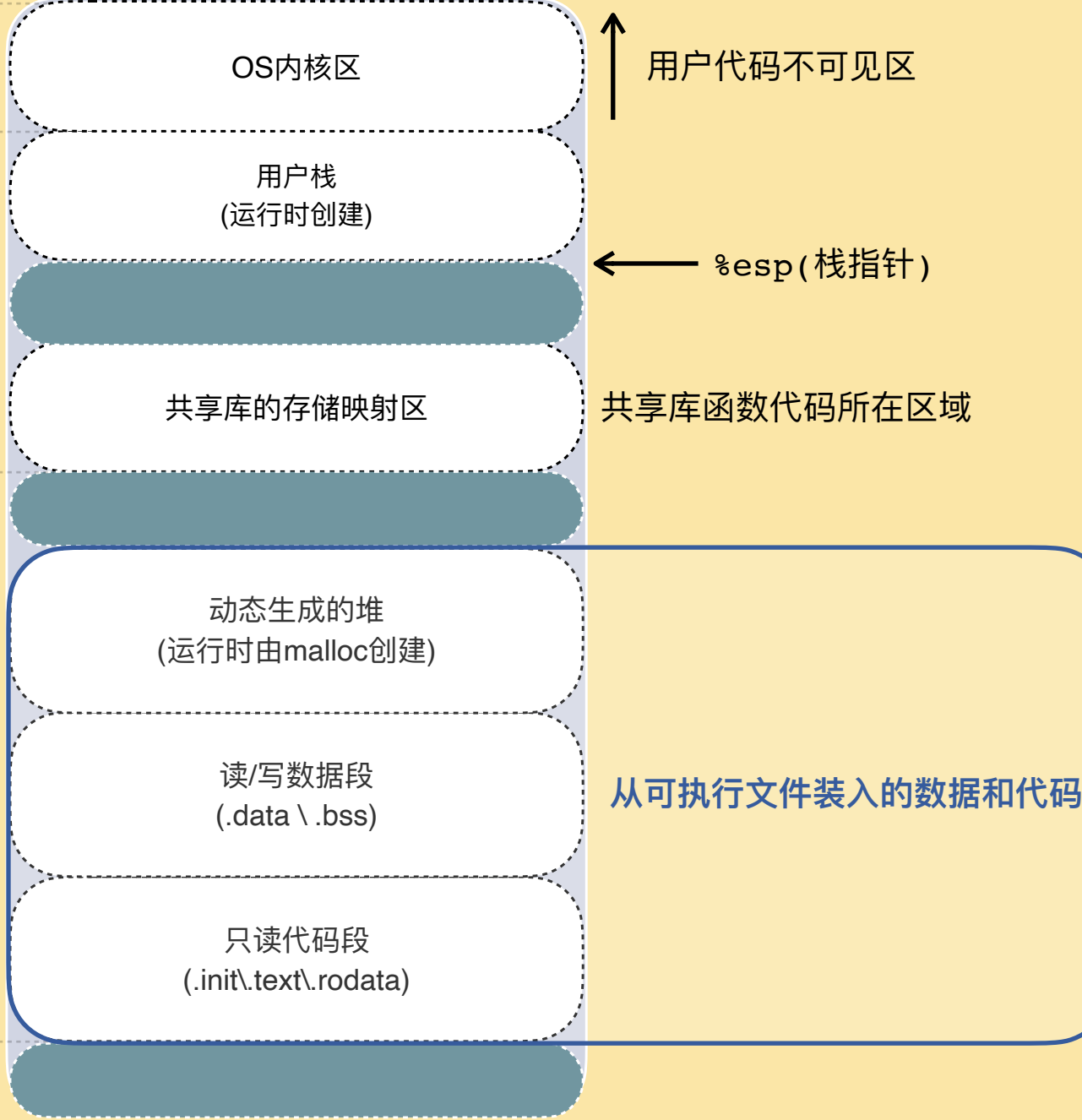
栈：用来实现**函数调用**，从用户空间的**最大地址向低地址**方向增长

0xFFFFFFFF

0xC0000000

0x40000000

0x08048000



内存中的一个进程

0xFFFFFFFF

0xC0000000

操作系统内核区：将操作系统内核映射到这个区域，这个区域里的内容对进程**不可见**。对于虚拟存储系统中每个进程，它都以为整个系统中只有它自己和os内核

并且0xC000 0000以上是os内核的活动空间



内存中的一个进程

用户栈： 存局部变量，函数调用传参过程也会用到栈

0xFFFFFFFF

0xC0000000

0x40000000

0x08048000

OS内核区

用户栈
(运行时创建)

共享库的存储映射区

动态生成的堆
(运行时由malloc创建)

读/写数据段
(.data \ .bss)

只读代码段
(.init\text\rodata)

↑ 用户代码不可见区

← %esp (栈指针)

共享库函数代码所在区域

从可执行文件装入的数据和代码

内存中的一个进程

共享库的存储映射区： 将共享库 (如动态链接库DLL)
映射到进程的虚拟空间部分
使多个进程能共享同一个物理内存副本的库
节省了内存资源
映射区域允许进程执行库中的代码
并访问库中的数据
就像这些代码和数据是进程自己的一部分一样

0xFFFFFFFF
0xC0000000
0x00000000
0x08048000



内存中的一个进程

0xFFFFFFFF

0xC0000000

0x40000000

0x08048000

OS内核区

用户栈
(运行时创建)

共享库的存储映射区

动态生成的堆
(运行时由malloc创建)

读/写数据段
(.data \ .bss)

只读代码段
(.init\text\rodata)

↑ 用户代码不可见区

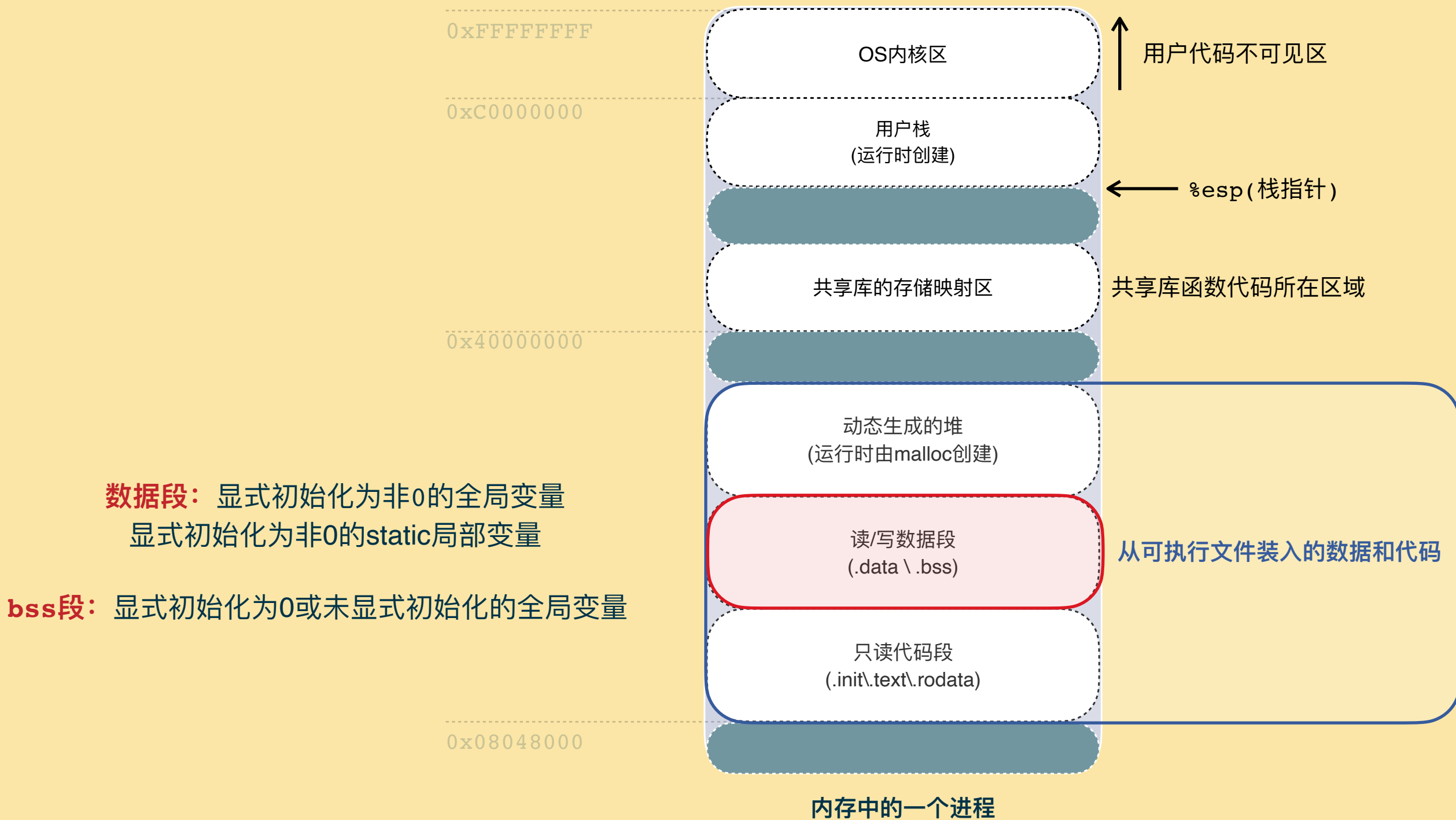
← %esp (栈指针)

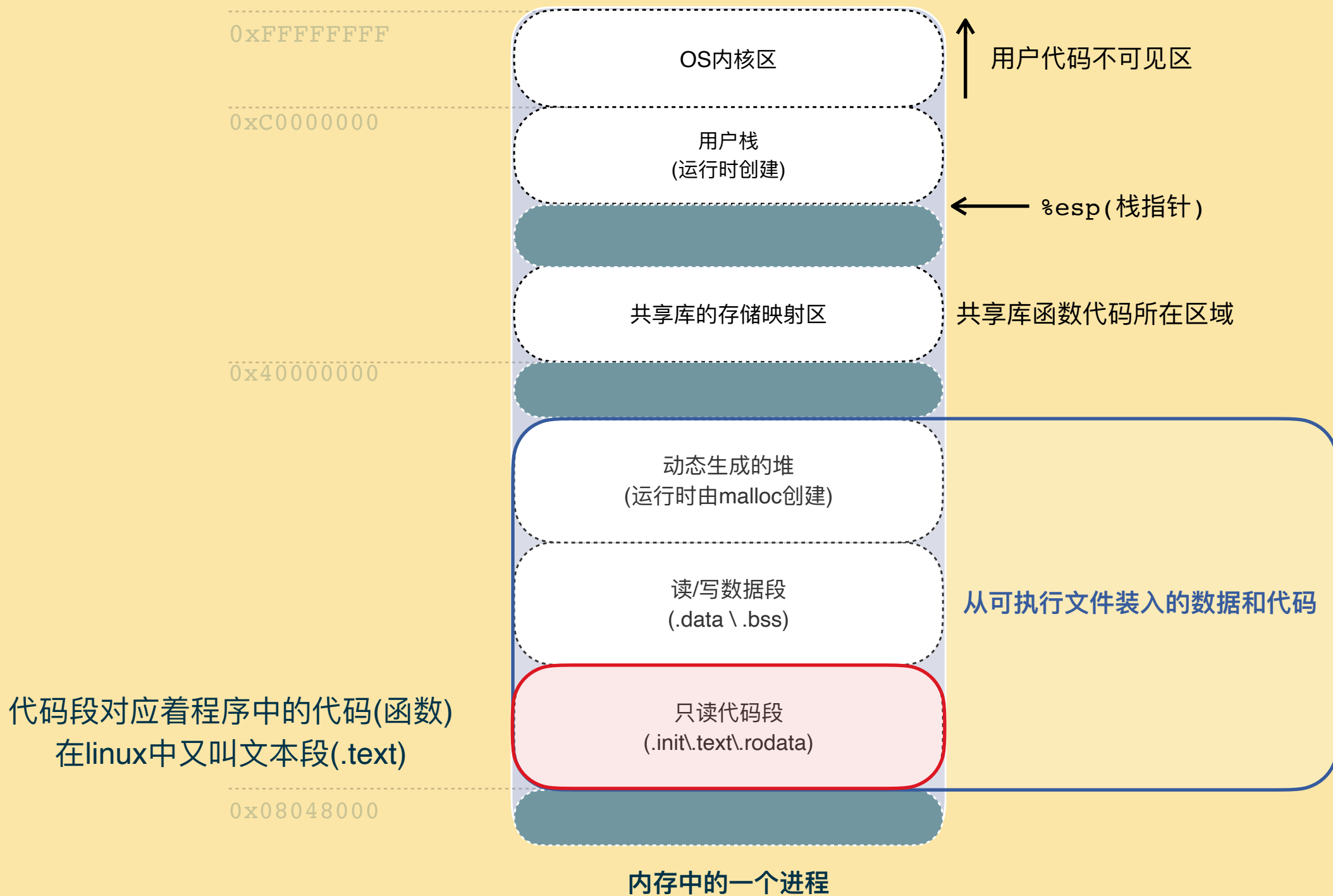
共享库函数代码所在区域

从可执行文件装入的数据和代码

堆： 进程根据需求判断是否使用堆
使用时自行使用malloc函数申请
结束后自行使用free函数释放

内存中的一个进程





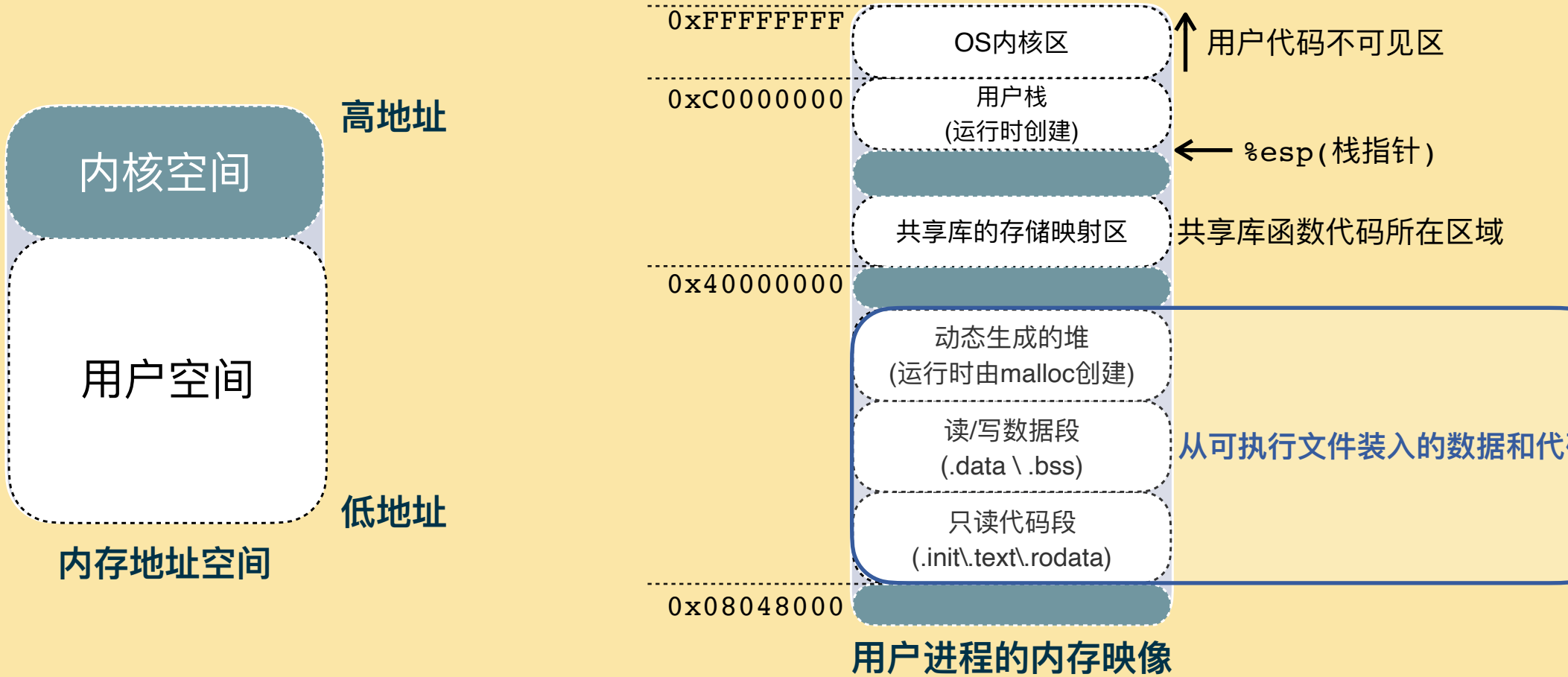


进程

- 基本概念
- 内存映像**
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

请务必学习完虚拟存储器后再观看本部分

虚拟化技术：每个进程访问自己的地址空间，os会以某种方式将它映射到机器的物理内存中。
对于每个进程，进程都认为自己独占了整个cpu和整个虚拟内存空间





进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程的三种基本状态

就绪态

进程已经准备好执行
已经分配到除了CPU的所有必要资源
只要获得CPU就可以立即执行
若有多个就绪态进程
通常会将其排成队列
称就绪队列

执行态

进程获得CPU后
程序“正在执行”这一状态
对任何一个时刻而言
在单处理机系统中
只有一个进程处于执行状态
在多处理机系统中
可能有多个进程处于执行状态

阻塞态

正在执行的进程由于发生某事件
(如io请求、申请缓冲区失败等)
暂时无法执行
即进程的执行受到了阻塞
这时会引发系统调度
OS把处理机分配给另一个就绪进程
让受阻进程处于暂停状态
这种暂停状态称为阻塞状态
处于阻塞状态的进程排成队列
叫阻塞队列



进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

进程的三种基本状态

就绪态

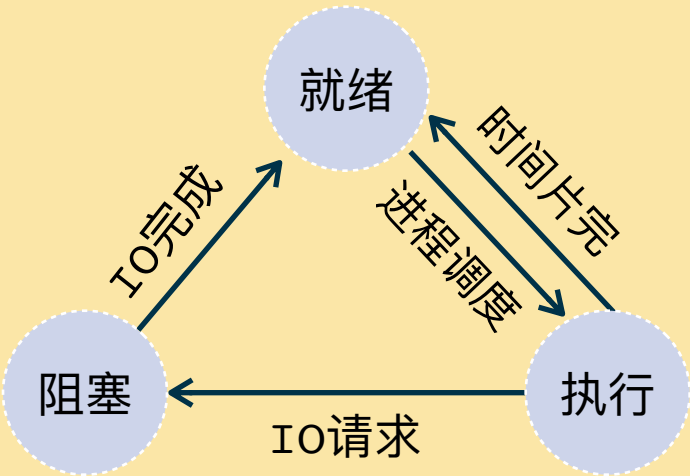
进程已经准备好执行
已经分配到除了CPU的所有必要资源
只要获得CPU就可以立即执行

执行态

进程获得CPU后
程序“正在执行”这一状态

阻塞态

正在执行的进程由于发生某事件
(如io请求、申请缓冲区失败等)
暂时无法执行
即进程的执行受到了阻塞





进程

[基本概念](#)[内存映像](#)[状态&转换](#)[进程管理的数据结构](#)[控制](#)[通信](#)

创建状态

创建进程是一个复杂的过程，一般通过多个步骤才能完成

- 1.由进程申请一个空白PCB，并且向PCB中填写用于控制和管理进程的信息
- 2.为该进程分配运行时所必需的资源
- 3.把该进程状态转换为就绪态，插入就绪队列之中

若进程所必需的资源尚不能得到满足（如内存不足），创建工作尚未完成，此时称为创建状态



基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

创建状态

创建进程是一个复杂的过程，一般通过多个步骤才能完成

两个创建进程相关的系统调用：

fork()：父进程调用fork()会**拷贝**出一个子进程，它拥有自己的地址空间、寄存器、PC等，fork()返回给父进程子进程的PID(**内部标识符，后面会讲**)

exec()：从可执行程序中加载代码和静态数据，并用它覆写自己的代码段，堆栈等其他内存空间也会被重新初始化

先fork()，再exec()，就拥有了新的进程



基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

终止状态

进程的终止需要两个步骤

1. 等待OS进行善后处理

2. 将进程PCB清零，将PCB空间返还给OS

当一个进程到达了自然结束点，或者出现了无法克服的错误，或被os所终止，或被其它有终止权的进程终止时，它就会进入终止状态。进入终止状态的进程不会被执行，但是os中会保留一个记录，保存状态码和一些计时统计数据供其它进程收集

其它进程完成信息提取后，系统就会删除该进程，PCB清零并将空白PCB还给OS



基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

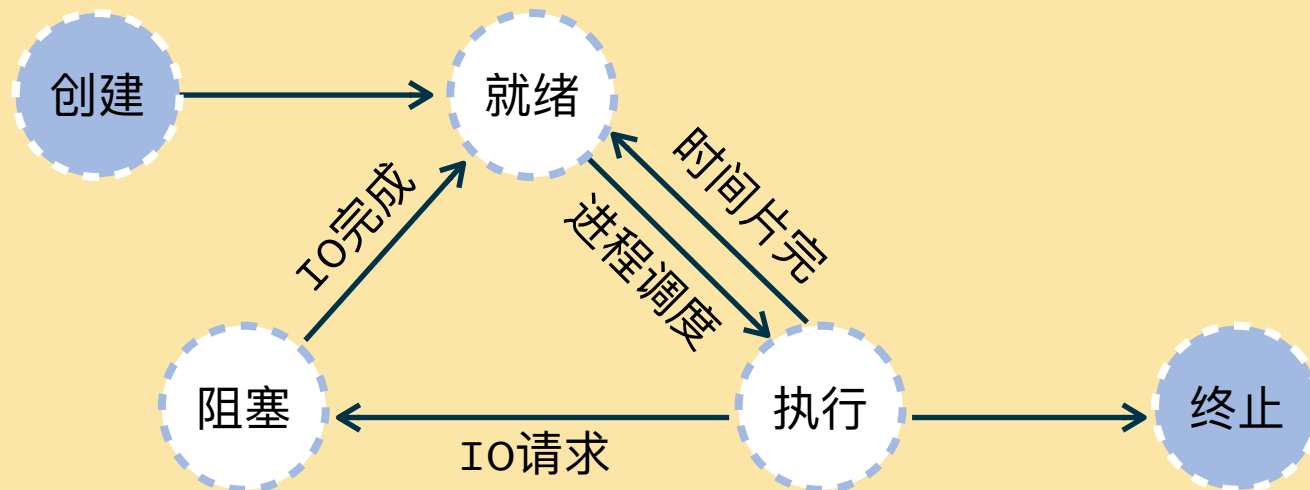
创建 & 终止

创建进程是一个复杂的过程，一般通过多个步骤才能完成

1. 由进程申请一个空白PCB，并且向PCB中填写用于控制和管理进程的信息
2. 为该进程分配运行时所必需的资源
3. 把该进程状态转换为就绪态，插入就绪队列之中

进程的终止也要通过两个步骤

1. 等待OS进行善后处理
2. 将进程PCB清零，将PCB空间返还给OS



进程



基本概念

内存映像

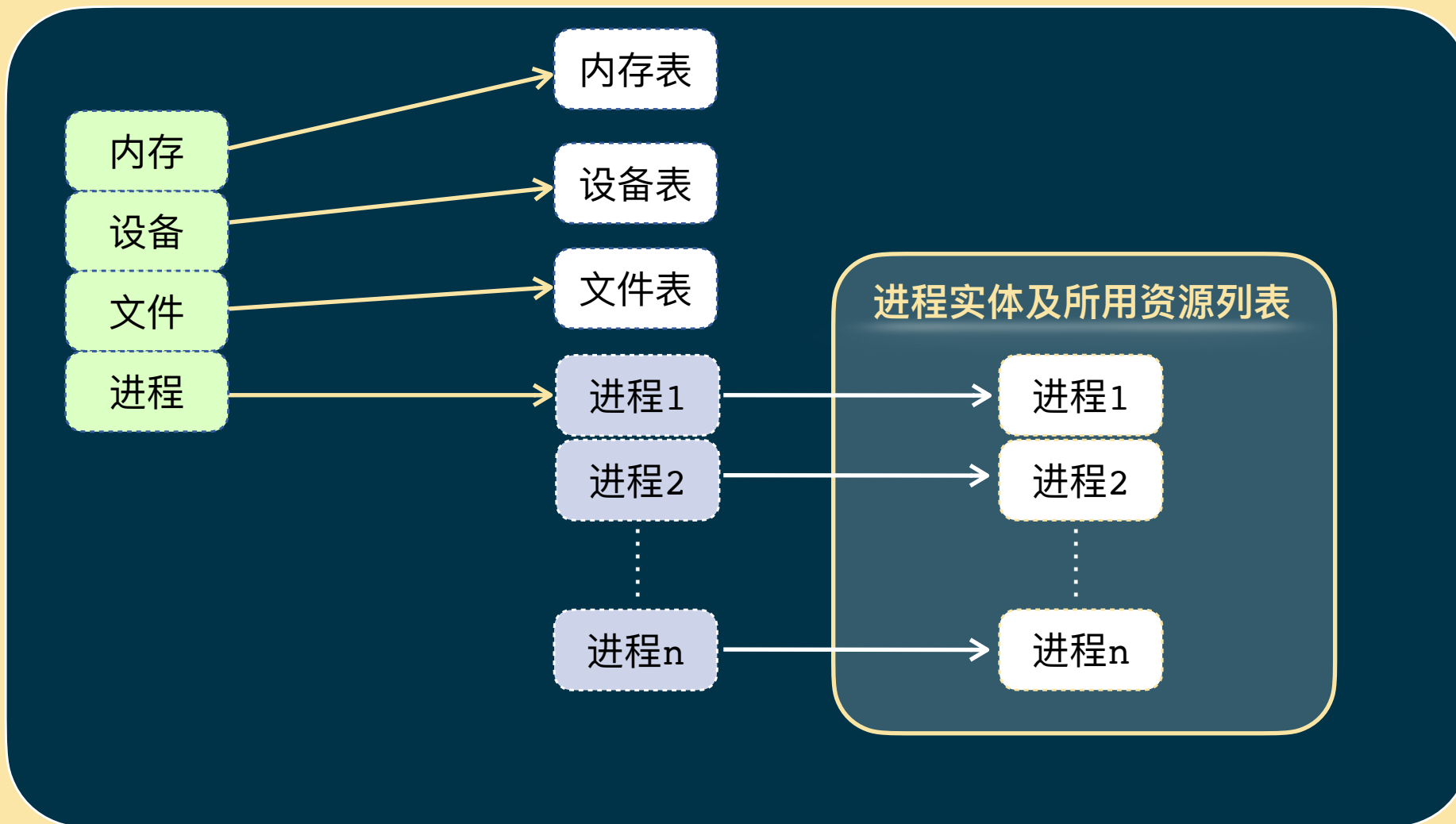
状态&转换

进程管理的数据结构

控制

通信

为了便于管理和使用计算机中的各类资源，os将它们抽象为相应的各种数据结构，并提供一组对资源进行操作的命令





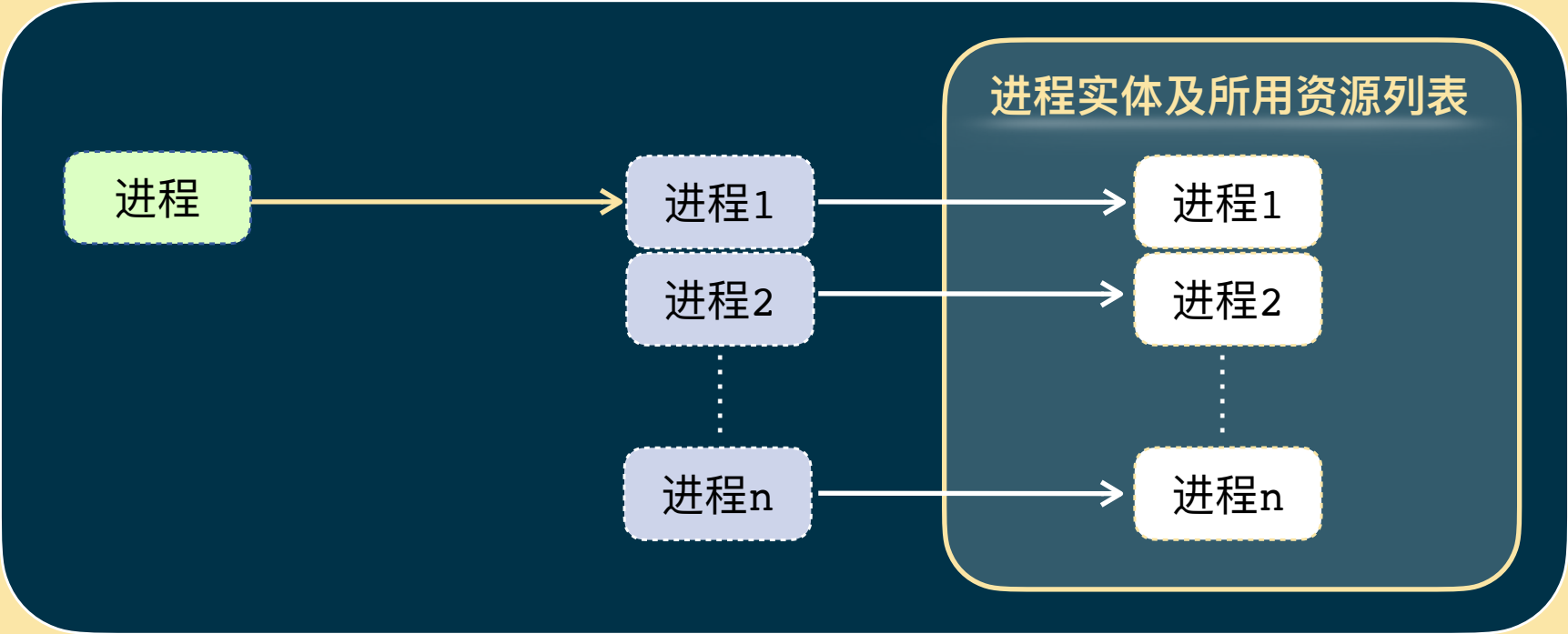
进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

为了便于管理和使用计算机中的各类资源，os将它们抽象为相应的各种数据结构，并提供一组对资源进行操作的命令

为了便于系统描述和管理进程，os为每个进程定义了一个数据结构**PCB (进程控制块)**

PCB用于记录os所需的、用于描述进程当前情况以及管理进程运行状态的全部信息是os中最重要的记录型数据结构





基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

为了便于系统描述和管理进程，os为每个进程定义了一个数据结构**PCB (进程控制块)**
PCB用于记录os所需的、用于描述进程当前情况以及管理进程运行状态的全部信息
是os中最重要的记录型数据结构

1. 作为独立运行基本单位的标志

一个程序（含数据）配置了PCB后
就表示它已经是一个能在多道程序环境下独立运行的基本单位了
创建新进程时会建立一个PCB；进程结束时会回收其PCB，进程也随之消亡
OS通过PCB来感知进程的存在，也是进程存在于系统中的唯一标志

2. 实现间断性运行方式

多道程序环境下，程序是停停走走的，进程被阻塞的时候必须保留自己运行时CPU的现场信息
再次被调度运行时需要恢复CPU现场信息
有了PCB，系统就可以把这些信息保存在其中

3. 提供进程管理所需要的信息

当调度程序调度到某进程时，只能根据该进程PCB中记录的程序和数据在内存/外存的起始地址中找到相应程序/数据
进程需要访问文件或者IO时，也需要借助PCB中信息，还可根据PCB了解该进程所需全部资源等信息

4. 提供进程调度所需信息

PCB中有进程所处状态等信息，也会根据需要记录优先级、等待时间、已执行时间等信息

5. 实现与其它进程的同步和通信

PCB中还有用于实现进程通信的区域或通信队列的指针等



进程

[基本概念](#)[内存映像](#)[状态&转换](#)[进程管理的数据结构](#)[控制](#)[通信](#)

PCB中的信息

进程标识符

外部标识符

为方便用户对进程的访问，为每个进程设置一个外部标识符
通常由字母和数字组成。为了描述进程家族关系，还应设置父进程标识符和子进程标识符
此外还可设置用户标识符，指示拥有该进程的用户

内部标识符

为方便系统对进程的使用，在os中又为进程设置了内部标识符
赋予每个进程唯一一个数字标识符，通常是一个进程序号



进程

[基本概念](#)[内存映像](#)[状态&转换](#)[进程管理的数据结构](#)[控制](#)[通信](#)

处理机状态

处理机的状态信息，也称处理机的上下文，主要由处理机的各种寄存器中的内容组成

1. 通用寄存器 (用户可视寄存器)： 可被用户程序访问，用于暂存信息

2. 程序计数器： 其中存放了要访问下一条指令的地址

3. 程序状态字寄存器： 每个用户进程都有与之相关的系统栈，用于存放进程和系统的调用参数和调用地址
栈指针指向该栈栈顶。处理机执行状态时，正在处理的许多信息也是放在寄存器中的
进程被切换时，处理机状态信息都必须保存在相应PCB中

4. 堆栈指针寄存器： 指向用户自己堆栈的指针和指向内核栈的指针



进程

[基本概念](#)[内存映像](#)[状态&转换](#)[进程管理的数据结构](#)[控制](#)[通信](#)

进程调度信息

因为os要对进程进行调度，所以需要了解必要的信息

- 1. 进程状态：**指明进程当前的状态(阻塞、就绪、运行)，作为进程调度、对换的依据
- 2. 进程优先级：**描述进程使用处理机的优先级别（用一个整数表示），优先级高的优先获得处理机
- 3. 进程调度所需的其它信息：**部分进程调度算法需要知道进程已等待CPU时间总和、进程已执行时间总和等
- 4. 事件：**进程由执行状态转换为阻塞状态所等待发生的事件（阻塞的原因）



进程

[基本概念](#)[内存映像](#)[状态&转换](#)[进程管理的数据结构](#)[控制](#)[通信](#)

进程控制信息

用于进程控制所必需的信息

- 1. 程序和数据地址：**即程序和数据的内存或外存起始地址，便于该进程执行时能从PCB中快速找到程序和数据
- 2. 进程同步和通信机制：**比如消息队列指针、信号量等，它们可能会全部或部分放在PCB中
- 3. 资源清单：**列出了该进程在运行期间所需的全部资源（除CPU外）
- 4. 链接指针：**给出本进程所在队列中的下一个进程的PCB起始地址



基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

PCB中的信息

PCB 记录操作系统描述进程当前情况、管理进程运行状态所需的全部信息。

01

进程标识符

外部标识符

内部标识符

02

处理机状态

通用寄存器

指令计数器

程序状态字寄存器

03

进程调度信息

进程状态

进程优先级

其他信息

事件

04

进程控制信息

程序&数据地址

资源清单

进程同步和通信机制

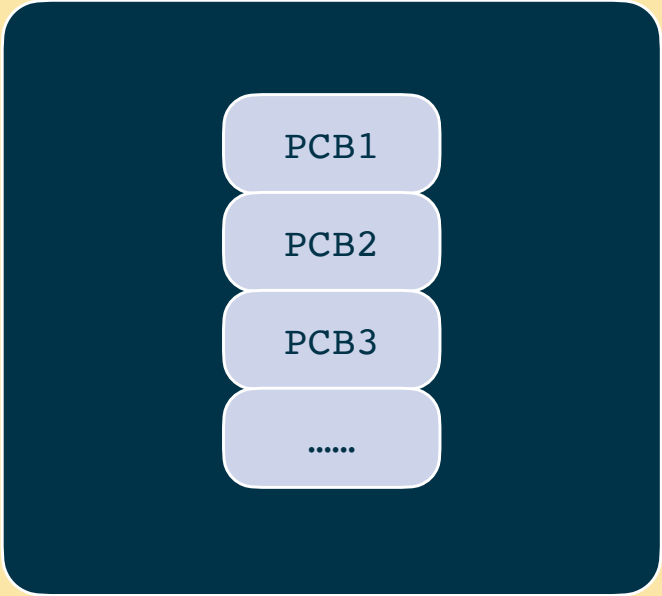
链接指针

进程

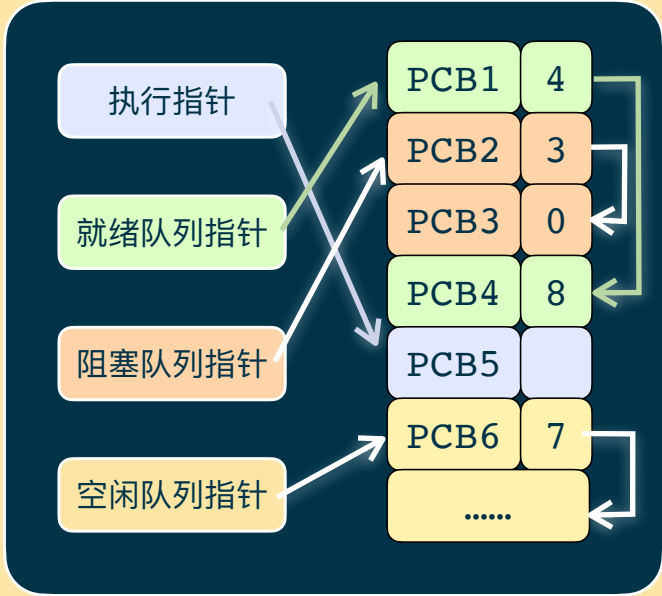
- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

PCB的组织方式

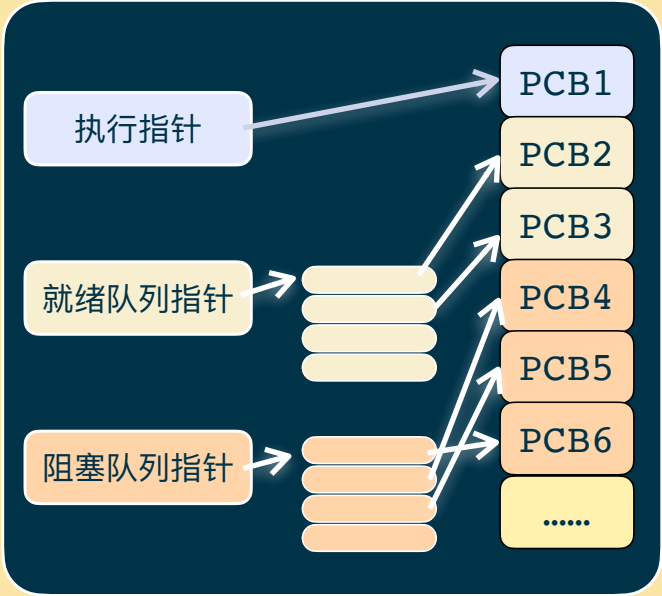
- 线性方式
- 链接方式
- 索引方式



所有PCB组织在一张线性表中
起始地址放在内存的一个专用区域中
这样的方式简单开销小
但每次查找都需要扫描整张表
适合进程不多的系统



将具有相同状态的进程PCB分别链接成一个队列
就绪队列可按照优先级排序
阻塞队列可以根据阻塞原因分成多个阻塞队列



OS根据进程状态的不同
建立几张索引表
把各索引表在内存中起始地址
记录在内存的专用单元中
每个索引表的表目中
记录具有相应状态的某个PCB在PCB表中的地址



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

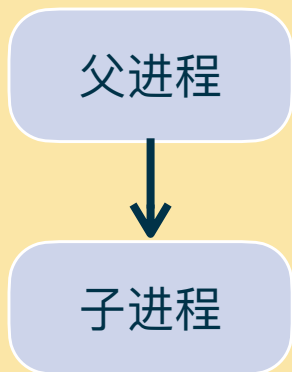
通信

进程的创建

os中允许一个进程创建另一个进程，通常把创建进程的进程称为父进程，被创建的称为子进程

子进程可以继续创建其自己的子进程（即父进程的孙进程）

如在UNIX系统中，进程与其子孙进程可以共同组成一个进程家族（进程组）



子进程可以继承父进程所拥有的资源（打开的文件，分配的缓冲区）

当子进程被撤销时，应将从父进程那里获得的资源归还父进程。

部分系统中，当一个进程终止时，其所有子孙进程也必须随之终止

这种机制称**级联终止**

并非所有操作系统都采用这一策略

在windows系统中不存在任何进程层次结构的概念，所有进程有相同的地位

一个进程创建了另外一个进程后，创建进程获得一个句柄，作用相当于令牌

可控制被创建进程，同时句柄可以进行传递

因此进程间关系不是层次关系，而是控制与被控制的简单关系



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

引起进程创建的事件

01

用户登录

分时系统中，用户在终端输入登录命令后，若登录成功，则系统会为该用户创建一个进程，并插入就绪队列

02

作业调度

多道批处理系统中，作业调度程序按一定的算法调度到某个队列中

03

提供服务

运行中的用户程序提出某种请求后，系统专门创建一个进程为用户提供所需服务
(如打印文件创建打印进程)

04

应用请求

上面三种情况是os内核为用户创建新进程；针对应用请求则是用户进程自己创建新进程，进程间并发执行完成任务



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程创建的步骤

1. 申请空白PCB，为新进程申请一个唯一的数字标识符
2. 为新进程分配其运行所需的资源，包括各种物理和逻辑资源，如内存、文件、IO设备、CPU时间
3. 初始化PCB，包括以下步骤
 - 3-1 初始化标志信息，将系统分配的标识符和父进程标识符填入新PCB；
 - 3-2 初始化处理机状态信息，使程序计数器指向程序的入口地址，使栈指针指向栈顶
 - 3-3 初始化处理机控制信息，将进程的状态设置为就绪状态或静止就绪状态，通常还须将其设置为最低优先级
4. 如果进程就绪队列能够接纳新进程，就将新进程插入就绪队列



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程创建的步骤

进程创建新进程时，有两种执行的可能：

父进程与子进程并发执行

父进程等待，直到其某个或全部子进程执行完毕

新进程的内存映像也有两种可能

子进程是父进程的复制品（子进程与父进程具有相同程序和数据）

子进程加载另一个新程序



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程的终止

引起进程终止的操作

正常结束

进程的任务已完成
准备退出运行
任何系统中都应有一个用于表示进程已经运行完成的指示
通常会在程序的最后安排一条指令用于向os表示运行已结束
程序运行到该指令时
会产生一个中断
通知os本进程运行完毕

异常结束

指进程在运行时
发生了某种异常事件
使程序无法继续运行

保护错

指令错

特权指令错

运行超时

等待超时

算数运算错

IO错

外界干预

进程应外接请求而终止
(操作员或os干预, 如死锁)



基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程终止的过程

发生了要求终止进程的事件后，os会调用进程终止原语，按照一定步骤终止指定进程

1. 根据被终止进程的标识符，从PCB集合中找到该进程PCB，读出该进程状态
2. 若被终止进程处于执行状态，立刻中止执行
3. 若该进程还有子孙进程，还应终止其所有子孙进程，以防它们称为不可控的进程
(不是所有的操作系统都会这样做！)
4. 将被终止进程拥有的全部资源归还给父进程或系统
5. 将被终止进程的PCB从所在队列中移除



基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

引起进程阻塞的事件

01 向os请求共享资源失败

(请求打印机, 但是已经没可分配打印机了)

02 等待某种操作的完成

(请求IO设备, 但要拿到IO设备进行IO操作的结果才能继续执行)

03 新数据尚未到达

(对于合作进程, 一个进程需要另一进程的数据才能处理, 而该数据尚未到达)

04 等待新任务的到达

(某些os中会设置特定系统进程, 完成任务后自我阻塞以等待新任务到来)



基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

进程阻塞的过程

正在执行的进程，发生引起阻塞的事件，就会调用阻塞原语block将自己阻塞
故阻塞是进程自身的主动行为

进入block阶段后，由于该进程还处在执行状态，OS停止执行该进程，PCB中改为阻塞态
将PCB插入阻塞队列，再转至调度程序进行重新调度操作

阻塞原语和唤醒原语是一对作用相反的原语，使用它们时必须成对使用

进程唤醒的过程

被阻塞进程所期待的事件发生时，就会调用唤醒原语wakeup以将等待该事件的进程唤醒

wakeup的执行过程

1. 将被阻塞的进程从等待该事件的阻塞队列中移出，将其PCB中的现行状态改为就绪
2. 将该PCB插入就绪队列



进程

[基本概念](#)[内存映像](#)[状态&转换](#)[进程管理的数据结构](#)[控制](#)[通信](#)

进程通信（IPC）是指进程之间的信息交换
根据通信效率和数据量的不同，可分为低级通信和高级通信两类

低级通信

低级通信主要用于传递控制信息（如同步、互斥）
典型代表是PV操作

PV操作
(第二章爆学)

高级通信

用于高效传输大量数据
主要包括共享存储、消息传递和管道通信、信号等方式

[共享存储](#)[消息传递](#)[管道通信](#)

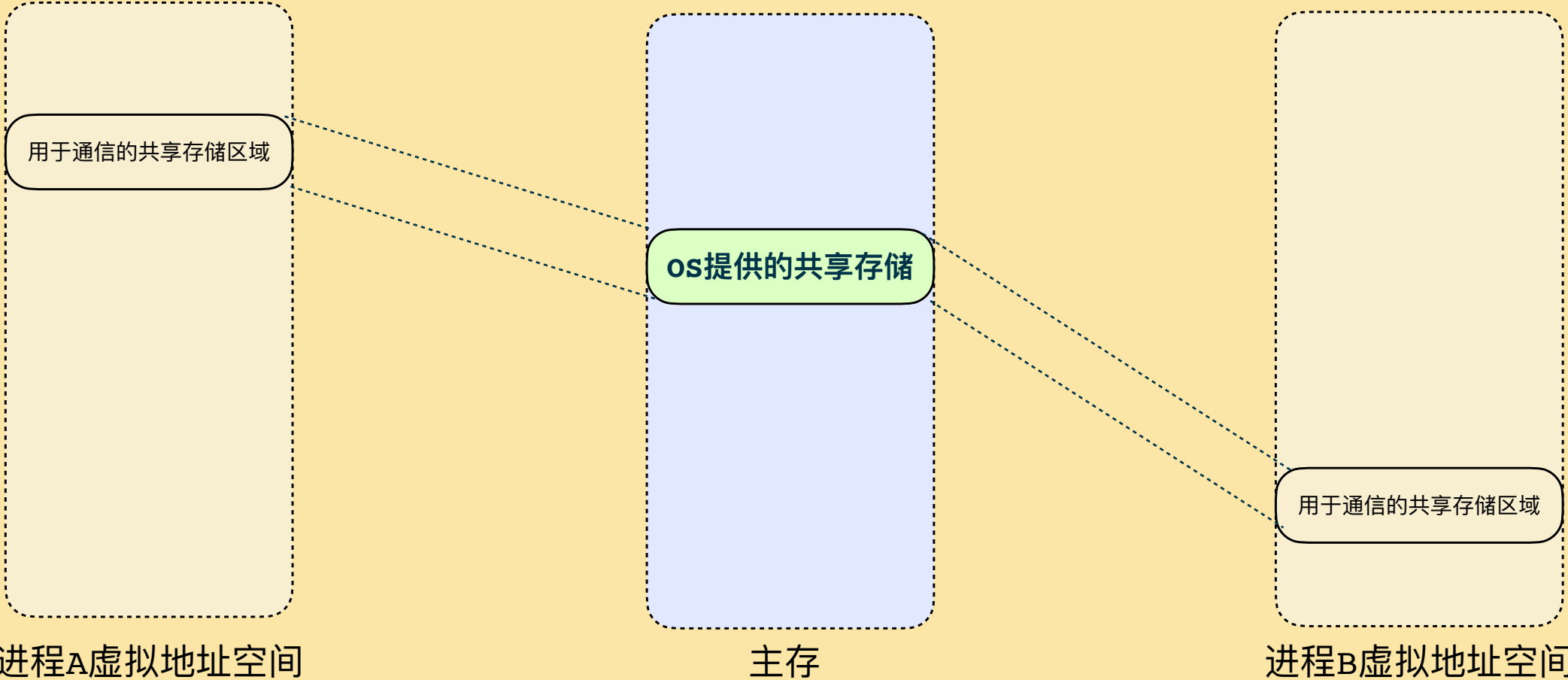


进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信**

共享存储

创建独立内存区域，映射到多个进程的虚拟地址空间
让多个进程能读写同一块内存区域





进程

基本概念

内存映像

状态&转换

进程管理的数据结构

控制

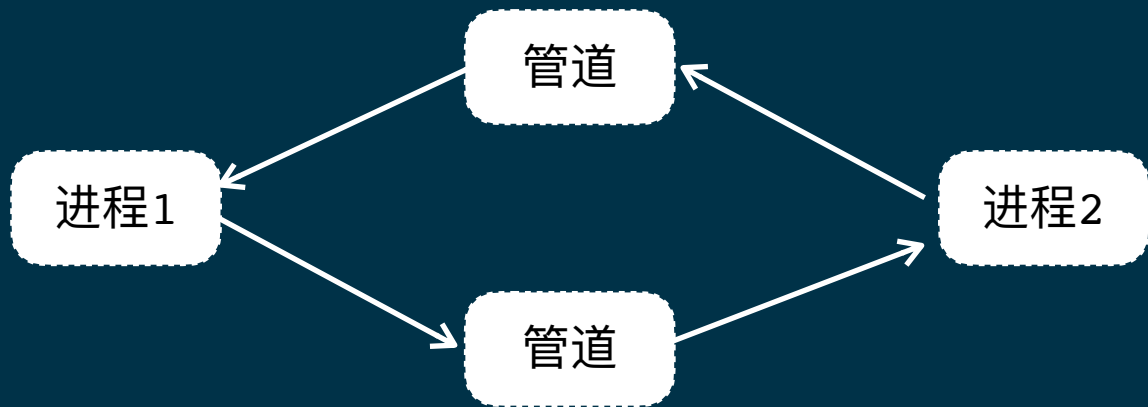
通信

管道通信

“管道”指用于连接一个读进程和一个写进程以实现它们之间通信的一个共享文件，也叫pipe文件

管道必须提供的3方面协调能力

1. **互斥**：当一个进程正在对管道执行读/写操作时，其它进程必须等待
2. **同步**：当写进程把一定数量的数据写入管道后，便等待直到读进程取走数据后，再把它唤醒
读进程读一空管道时，也应等待
3. 确定对方是否**存在**，对方已存在时才能进行通信





进程

[基本概念](#)[内存映像](#)[状态&转换](#)[进程管理的数据结构](#)[控制](#)[通信](#)

消息传递系统

消息传递系统中，进程不必借助任何共享数据结构或存储区，而是会以**格式化的消息**为单位，将通信的数据封装在消息中

1. **直接通信方式**：发送进程利用os所提供的发送原语，直接把消息发送给目标进程

2. **间接通信方式**：发送进程和接受进程都通过共享中间实体（信箱）进行消息的发送和接受

计网中会详细了解！



进程

基本概念

内存映像

状态&转换

进程管理的数据结构

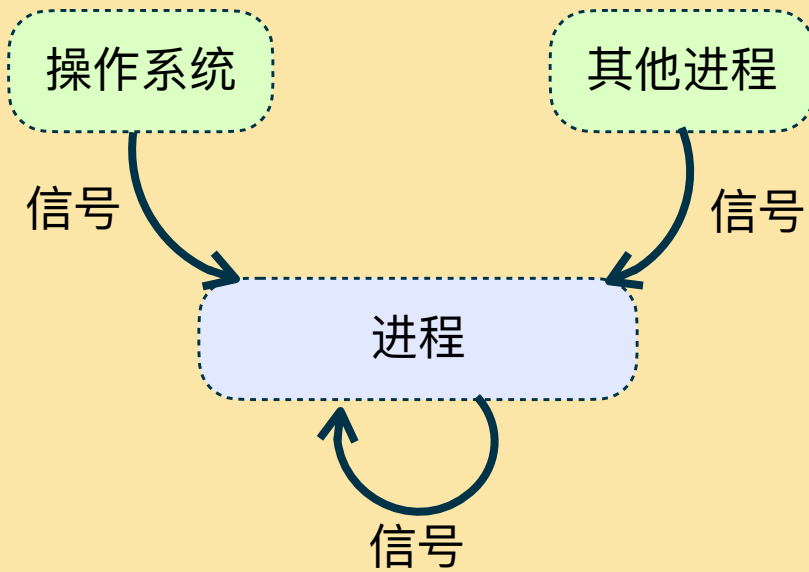
控制

通信

信号 (考纲新增内容)

用于通知进程事件的发生
不同系统事件对应不同信号类型

信号可以由操作系统内核或进程发送
可以由其他进程发送，也可以进程发送给自己



进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

信号 (考纲新增内容)

用于通知进程事件的发生
不同系统事件对应不同信号类型

操作系统

其他进程

进程

信号

信号

数组下标	信号	原因
0	SIGABRT	进程中止且强迫核心转储
1	SIGALRM	定时器超时
2	SIGFPE	出现浮点错误 (比如, 除0)
3	SIGHUP	进程所使用的电话线被挂断
4	SIGILL	用户按了DEL键中断了进程
5	SIGQUIT	用户按键要求核心转储
6	SIGKILL	杀死进程 (不能被捕捉或忽略)
7	SIGPIPE	进程写入了无读者的管道
8	SIGSEGV	进程引用了非法的内存地址
9	SIGTERM	用于要求进程正常终止
10	SIGUSR1	用于应用程序定义的目的
11	SIGUSR2	用于应用程序定义的目的

[0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 1 , 0]
signal[0] signal[1] signal[2] signal[3] signal[4] signal[5] signal[6] signal[7] signal[8] signal[9] signal[10] signal[11]

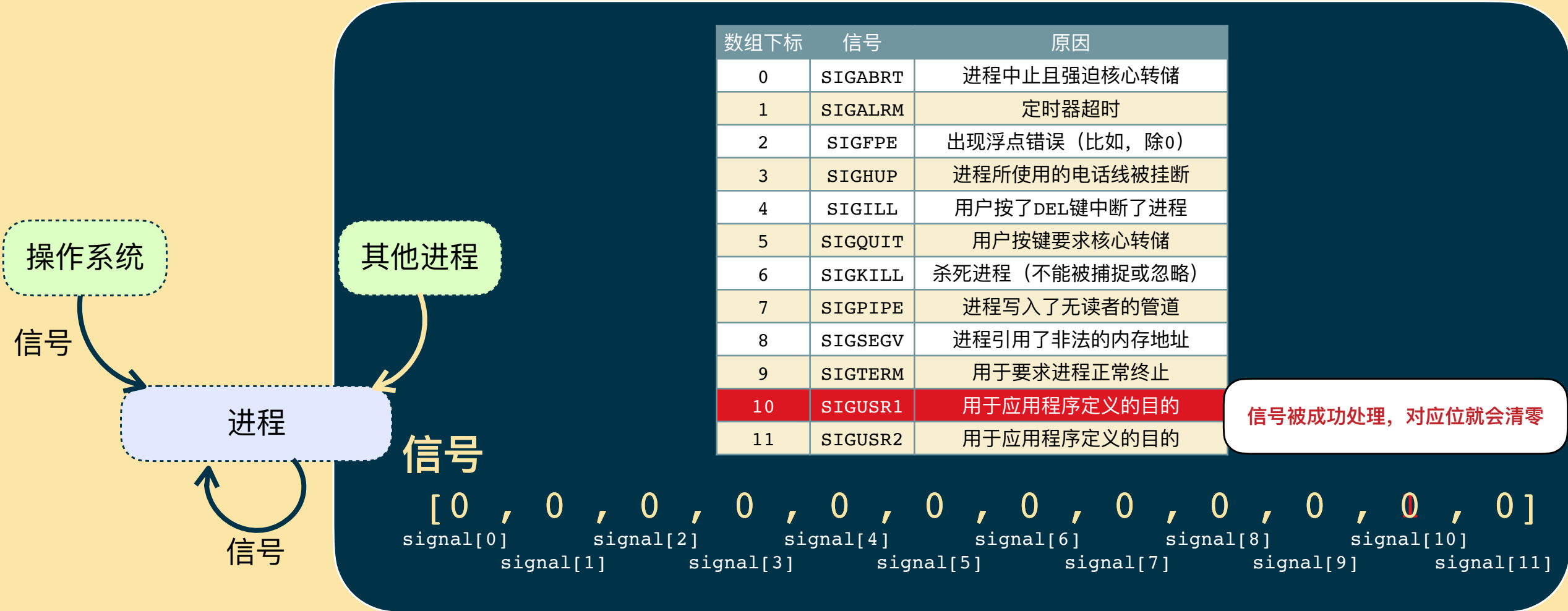
信号被成功处理, 对应位就会清零

进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

信号 (考纲新增内容)

用于通知进程事件的发生
不同系统事件对应不同信号类型



进程

- 基本概念
- 内存映像
- 状态&转换
- 进程管理的数据结构
- 控制
- 通信

信号 (考纲新增内容)

用于通知进程事件的发生
不同系统事件对应不同信号类型





基本概念

内存映像

状态&转换

进程管理的数据结构

控制

通信

信号 (考纲新增内容)

用于通知进程事件的发生

不同系统事件对应不同信号类型

数组下标	信号	原因
0	SIGABRT	进程中止且强迫核心转储
1	SIGALRM	定时器超时
2	SIGFPE	出现浮点错误 (比如, 除0)
3	SIGHUP	进程所使用的电话线被挂断
4	SIGILL	用户按了DEL键中断了进程
5	SIGQUIT	用户按键要求核心转储
6	SIGKILL	杀死进程 (不能被捕捉或忽略)
7	SIGPIPE	进程写入了无读者的管道
8	SIGSEGV	进程引用了非法的内存地址
9	SIGTERM	用于要求进程正常终止
10	SIGUSR1	用于应用程序定义的目的
11	SIGUSR2	用于应用程序定义的目的

信号的两种处理方式

执行默认信号处理程序

os预设了每类信号操作
直接照做

执行进程自定义信号处理程序

进程为信号自己写了处理函数
按照自己意图操作

信号 [0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 1 , 0]

阻塞 [0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 0 , 1 , 0]



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 【2014统考真题】下列关于管道 (Pipe) 通信的叙述中, 正确的是 () 。

- A. 一个管道可实现双向数据传输
- B. 管道的容量仅受磁盘容量大小限制
- C. 进程对管道进行读操作和写操作都可能被阻塞
- D. 一个管道只能有一个读进程或一个写进程对其操作



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

1. 【2014统考真题】下列关于管道 (Pipe) 通信的叙述中, 正确的是 () 。

A. 一个管道可实现双向数据传输

B. 管道的容量仅受磁盘容量大小限制

C. 进程对管道进行读操作和写操作都可能被阻塞

D. 一个管道只能有一个读进程或一个写进程对其操作



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2. 【2015统考真题】 下列选项中，会导致进程从运行态变为就绪态的事件是（）。

A. 执行P (wait) 操作

B. 申请内存失败

C. 启动I/O设备

D. 被高优先级进程抢占



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

2. 【2015统考真题】 下列选项中，会导致进程从运行态变为就绪态的事件是（）。

A. 执行P (wait) 操作

B. 申请内存失败

C. 启动I/O设备

D. 被高优先级进程抢占



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

3. 【2020统考真题】下列关于父进程与子进程的叙述中，错误的是（）。

- A. 父进程与子进程可以并发执行
- B. 父进程与子进程共享虚拟地址空间
- C. 父进程与子进程有不同的进程控制块
- D. 父进程与子进程不能同时使用同一临界资源



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

3. 【2020统考真题】下列关于父进程与子进程的叙述中，错误的是（）。

- A. 父进程与子进程可以并发执行
- B. 父进程与子进程共享虚拟地址空间**
- C. 父进程与子进程有不同的进程控制块
- D. 父进程与子进程不能同时使用同一临界资源



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

4. 【2024统考真题】下列选项中，操作系统在终止进程时不一定执行的是（）。

- A. 终止子进程
- B. 回收进程占用的设备
- C. 释放进程控制块
- D. 回收为进程分配的内存



真题训练

*此模块题目取自真题及与真题风格类似的模拟题

4. 【2024统考真题】下列选项中，操作系统在终止进程时不一定执行的是（）。

A. 终止子进程

B. 回收进程占用的设备

C. 释放进程控制块

D. 回收为进程分配的内存